

Homework2_DSBD

Ingegneria Informatica LM-32

Anno: 2025-2026

Giuseppe Ravesi

Sommario

1 Introduzione	3
2 Architettura del Sistema	3
2.1 Microservices Layer.....	3
2.2 API Gateway	4
2.3 Persistence Layer	4
2.4 Kafka Layer e notifiche asincrone	4
3 Modifiche al modello dei dati e alle API REST.....	6
3.1 Estensione del modello dei dati	6
3.2 Modifiche alle API REST del Data Collector	6
3.2.1 Inserimento di un aeroporto di interesse	6
3.2.2 Aggiornamento delle soglie di un aeroporto	7
3.2.3 Eliminazione e lettura delle preferenze.....	8
4 Circuit Breaker e resilienza verso OpenSky Network	8
4.1 Motivazioni dell'adozione del Circuit Breaker.....	8
4.2 Funzionamento del Circuit Breaker	9
4.3 Integrazione nel Data Collector	9
5 Meccanismo asincrono di notifica basato su Kafka.....	10
5.1 Ruolo di Kafka nell'architettura	10
5.2 Topic Kafka utilizzati.....	11
5.3 Data Collector come Kafka Producer	11
5.4 Alert System come Consumer e Producer.....	12
5.5 Alert Notifier System come Kafka Consumer	12
5.6 Diagramma di sequenza del sistema Kafka	13
5.7 Configurazione del servizio di notifica tramite SMTP	14
Conclusioni	15

1 Introduzione

Il presente documento descrive le estensioni architetturali e funzionali introdotte nel sistema sviluppato durante l'Homework 1, con l'obiettivo di migliorarne la resilienza, la scalabilità e la capacità di gestione di notifiche asincrone.

In particolare, Homework 2 introduce un meccanismo di **Circuit Breaker** per proteggere le comunicazioni verso servizi esterni, un **sistema di notifica asincrona** basato su **Apache Kafka** e un **API Gateway** come punto di ingresso unico al sistema. Tali componenti permettono di disaccoppiare le varie fasi di elaborazione, ridurre l'impatto di errori esterni e migliorare l'organizzazione complessiva delle comunicazioni tra microservizi.

L'architettura di base del sistema, composta da microservizi indipendenti con database separati e comunicazioni REST e gRPC, rimane invariata rispetto a quella presentata in Homework 1. Le modifiche introdotte si integrano quindi con la soluzione esistente, estendendone le funzionalità senza alterarne i principi progettuali fondamentali.

Nel seguito del documento verranno descritte le nuove componenti architetturali, le modifiche apportate al modello dei dati e alle API REST, nonché il funzionamento del sistema di notifiche asincrone e dell'API Gateway.

2 Architettura del Sistema

L'architettura del sistema sviluppato in Homework 1 è basata su un approccio a **microservizi**, in cui ciascun componente è responsabile di una specifica funzionalità applicativa ed è isolato a livello di rete e di persistenza. Tale architettura è stata mantenuta anche in Homework 2, estendendola con nuovi componenti infrastrutturali e servizi dedicati alla gestione delle notifiche asincrone.

La Figura 1 mostra l'architettura complessiva del sistema, includendo sia i microservizi già presenti in Homework 1 sia le nuove componenti introdotte in questa estensione.

2.1 Microservices Layer

Il livello dei microservizi comprende i seguenti componenti principali:

- **User Manager**

Gestisce le informazioni sugli utenti e fornisce API REST per la registrazione e la gestione dei profili. Espone inoltre un'interfaccia gRPC utilizzata dagli altri microservizi per verificare l'esistenza degli utenti.

Questo servizio utilizza un database dedicato (**User DB**) e opera esclusivamente sulla rete user-network.

- **Data Collector**

È responsabile della gestione degli aeroporti di interesse degli utenti, della raccolta periodica dei dati di volo interrogando l'API esterna OpenSky Network e dell'aggiornamento del database dei voli (**Data DB**).

Il Data Collector comunica con lo User Manager tramite gRPC ed espone API REST per la gestione delle preferenze utente. Tutte le chiamate verso OpenSky Network sono protette da un meccanismo di Circuit Breaker.

2.2 API Gateway

Per Homework 2 è stato introdotto un API Gateway basato su NGINX, che rappresenta l'unico punto di ingresso per i client esterni (browser o Postman).

Il gateway instrada le richieste HTTP verso i microservizi interni sulla base del prefisso dell'URL (ad esempio /users/* e /collector/*), nascondendo la complessità interna del sistema e centralizzando la gestione delle comunicazioni in ingresso..

2.3 Persistence Layer

Ogni microservizio utilizza un database dedicato, secondo il principio di **database per servizio**:

- **User DB (PostgreSQL)** per il servizio User Manager;
- **Data DB (PostgreSQL)** per il servizio Data Collector, contenente sia i dati sui voli sia le informazioni sulle soglie (high-value, low-value) associate agli aeroporti di interesse.

I database sono isolati su reti Docker separate (user-network e data-network) e non sono accessibili direttamente dagli altri microservizi.

2.4 Kafka Layer e notifiche asincrone

Homework 2 introduce un livello dedicato alla comunicazione asincrona basato su **Apache Kafka**, eseguito come **componente esterno** rispetto allo stack principale dei microservizi.

Il flusso asincrono è strutturato come segue:

- Il **Data Collector** agisce come **producer**, pubblicando eventi sul topic to-alert-system al termine di ogni ciclo di raccolta dei dati.
- L'**Alert System** consuma tali eventi, verifica il superamento delle soglie configurate dagli utenti e, in caso positivo, produce nuovi messaggi sul topic to-notifier.
- L'**Alert Notifier System** consuma i messaggi dal topic to-notifier ed è responsabile dell'invio delle notifiche finali tramite SMTP.

Questa architettura consente di disaccoppiare completamente la fase di raccolta dei dati dalla fase di notifica, migliorando la scalabilità e riducendo l'impatto di eventuali errori nei servizi di notifica.

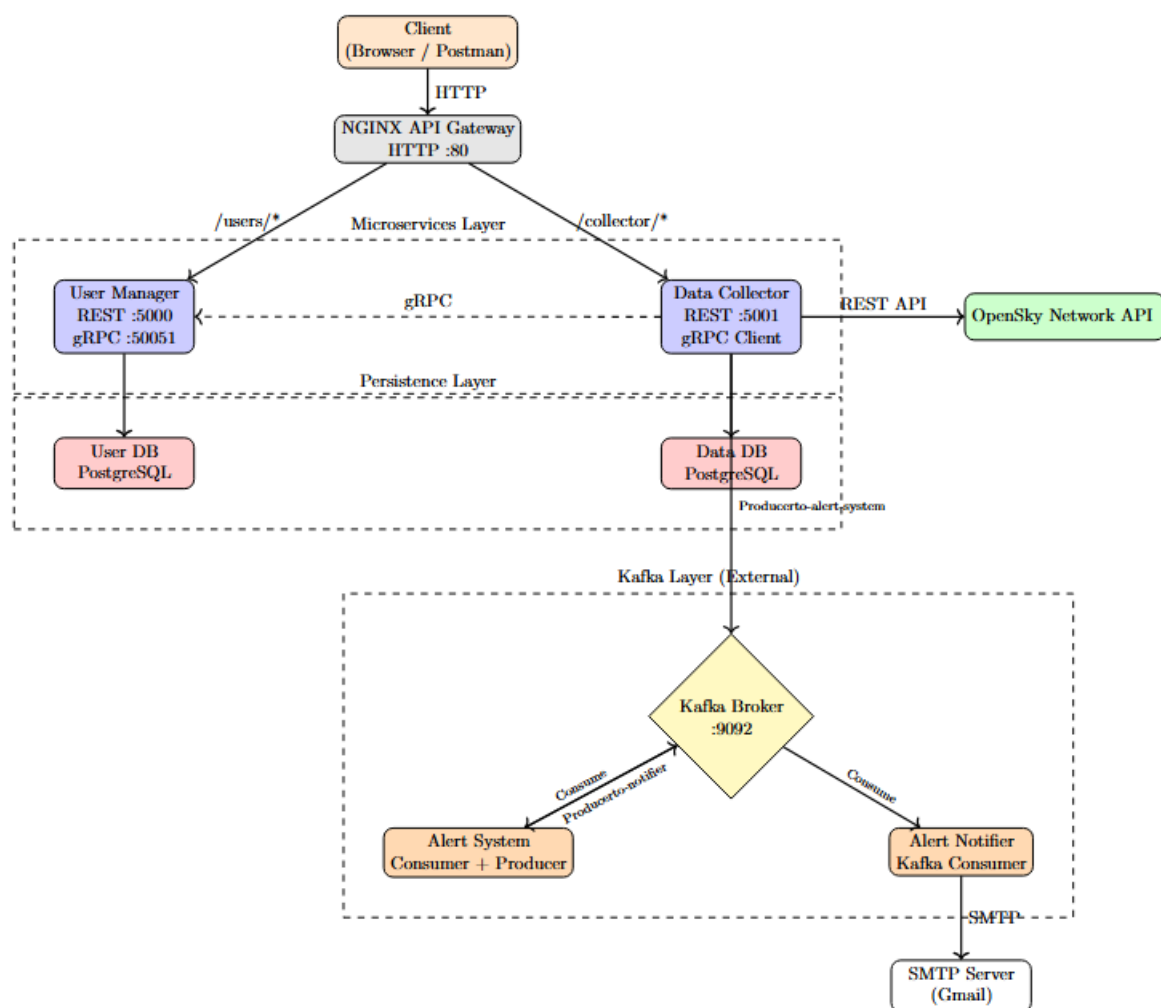


Figura 1 Homework 2 introduce un livello dedicato alla comunicazione asincrona basato su Apache Kafka, eseguito come componente esterno rispetto allo stack principale dei microservizi.

3 Modifiche al modello dei dati e alle API REST

In Homework 2 il sistema è stato esteso per supportare un meccanismo di notifica basato su soglie configurabili dall'utente.

Questa estensione ha richiesto modifiche sia al modello dei dati sia alle API REST già introdotte in Homework 1, mantenendo comunque la retrocompatibilità con le funzionalità esistenti.

3.1 Estensione del modello dei dati

Il modello dei dati del **Data Collector** è stato esteso per consentire la memorizzazione delle soglie associate a ciascun aeroporto di interesse di un utente.

In particolare, la tabella `airports` è stata modificata introducendo due nuovi attributi opzionali:

- **high_value**: soglia superiore sul numero di voli;
- **low_value**: soglia inferiore sul numero di voli.

Tali attributi presentano le seguenti caratteristiche:

- entrambi i valori sono opzionali;
- se entrambe le soglie sono presenti, deve valere la condizione **high_value > low_value**;
- i valori devono essere numeri interi non negativi.

Questa scelta progettuale consente di associare soglie personalizzate a ciascun aeroporto monitorato da un utente, senza introdurre nuove entità nel database e mantenendo una struttura dati compatta e facilmente interrogabile.

3.2 Modifiche alle API REST del Data Collector

Per supportare la gestione delle soglie, le API REST del Data Collector sono state estese e aggiornate come segue.

3.2.1 Inserimento di un aeroporto di interesse

L'endpoint per l'inserimento di un nuovo aeroporto è stato esteso per accettare i nuovi parametri `high_value` e `low_value`.

Endpoint

POST /airports

Payload (JSON)

```
{  
  
  "email": "user@example.com",  
  
  "airport_code": "LIRF",  
  
  "high_value": 200,  
  
  "low_value": 50  
}
```

I parametri relativi alle soglie sono opzionali e vengono validati lato server prima dell'inserimento nel database.

3.2.2 Aggiornamento delle soglie di un aeroporto

È stato introdotto un nuovo endpoint dedicato all'aggiornamento delle soglie associate a un aeroporto di interesse già registrato.

Endpoint

PUT /airports/preferences

Payload (JSON)

```
{  
  
  "email": "user@example.com",  
  
  "airport_code": "LIRF",  
  
  "high_value": 300,  
  
  "low_value": 100  
}
```

L'endpoint consente:

- l'aggiornamento di entrambe le soglie;

- l'aggiornamento di una sola soglia;
- la rimozione di una soglia impostandola a null.

Prima dell'aggiornamento vengono effettuati controlli di validazione sui valori forniti, in accordo con i vincoli definiti nel modello dei dati.

3.2.3 Eliminazione e lettura delle preferenze

Gli endpoint già presenti in Homework 1 per la lettura e la cancellazione degli aeroporti di interesse sono stati mantenuti invariati, ma ora restituiscono anche le informazioni relative alle soglie, se presenti.

Questa scelta garantisce la **retrocompatibilità** con i client esistenti e semplifica l'evoluzione dell'API.

4 Circuit Breaker e resilienza verso OpenSky Network

Il sistema sviluppato in Homework 2 interagisce con un servizio esterno, **OpenSky Network**, per il recupero delle informazioni sui voli in arrivo e in partenza dagli aeroporti di interesse. Poiché tale servizio esterno non è sotto il controllo dell'applicazione, risulta necessario introdurre un meccanismo di **protezione e resilienza** per evitare che malfunzionamenti o rallentamenti compromettano la stabilità complessiva del sistema.

Per questo motivo, tutte le operazioni verso OpenSky Network sono state protette tramite il pattern **Circuit Breaker**.

4.1 Motivazioni dell'adozione del Circuit Breaker

L'assenza di un meccanismo di protezione nelle chiamate verso servizi esterni può portare a diverse problematiche:

- blocco delle risorse del Data Collector in caso di timeout prolungati;
- propagazione degli errori verso altri microservizi;
- degrado delle prestazioni complessive del sistema;
- impossibilità di distinguere tra errori temporanei e malfunzionamenti persistenti.

L'introduzione del Circuit Breaker consente di:

- limitare il numero di chiamate verso un servizio instabile;
- prevenire sovraccarichi;
- garantire una risposta controllata anche in condizioni di errore.

4.2 Funzionamento del Circuit Breaker

Il Circuit Breaker implementato nel Data Collector segue il classico modello a **tre stati**:

- **CLOSED**
Le richieste verso OpenSky Network vengono effettuate normalmente.
Gli errori vengono monitorati per valutare la stabilità del servizio.
- **OPEN**
Il servizio esterno è considerato non disponibile.
Le chiamate verso OpenSky Network vengono immediatamente bloccate e non inviate.
- **HALF-OPEN**
Dopo un intervallo di tempo configurabile, il Circuit Breaker consente un numero limitato di richieste di prova.
In base all'esito:
 - successo → ritorno allo stato CLOSED;
 - fallimento → ritorno allo stato OPEN.

Questo comportamento permette al sistema di “testare” la ripresa del servizio esterno senza sovraccaricarlo.

4.3 Integrazione nel Data Collector

Nel progetto, il Circuit Breaker è stato implementato come **componente indipendente**, richiamato dal client responsabile della comunicazione con OpenSky Network.

Ogni chiamata al servizio OpenSky è quindi mediata dal Circuit Breaker, che:

1. verifica lo stato corrente del circuito;
2. decide se consentire o bloccare la richiesta;
3. aggiorna il proprio stato in base all'esito della chiamata.

Nel caso in cui il Circuit Breaker sia in stato **OPEN**, il Data Collector intercetta l'eccezione generata e procede con il recupero dei dati relativi agli altri aeroporti o utenti, evitando il fallimento dell'intero processo di raccolta.

5 Meccanismo asincrono di notifica basato su Kafka

Per soddisfare il requisito di notifica automatica al superamento di determinate soglie di traffico aeroportuale, il sistema è stato esteso introducendo un **meccanismo asincrono di comunicazione basato su Apache Kafka**.

L'adozione di un message broker consente di:

- disaccoppiare i microservizi coinvolti;
- evitare comunicazioni sincrone bloccanti;
- gestire in modo scalabile e affidabile la propagazione degli eventi;
- introdurre nuovi consumer senza modificare i producer esistenti.

Nel sistema sviluppato, Kafka rappresenta il **canale di comunicazione principale** tra i componenti responsabili della raccolta dati e quelli deputati alla generazione e all'invio delle notifiche.

5.1 Ruolo di Kafka nell'architettura

Kafka è stato integrato come **componente infrastrutturale esterno** rispetto al docker-compose principale dell'applicazione, ed è accessibile dai microservizi tramite una rete Docker condivisa.

All'interno dell'architettura, Kafka svolge il ruolo di:

- **event bus** per la propagazione degli aggiornamenti;
- **buffer persistente** per i messaggi prodotti;
- **meccanismo di disaccoppiamento temporale** tra producer e consumer.

Questa scelta consente al sistema di continuare a funzionare correttamente anche in presenza di:

- consumer temporaneamente non disponibili;
- ritardi nella gestione delle notifiche;
- picchi di eventi prodotti dal Data Collector.

5.2 Topic Kafka utilizzati

Nel progetto sono stati definiti due topic distinti, ciascuno con una responsabilità ben precisa:

- **to-alert-system**

Topic su cui il **Data Collector** pubblica gli eventi relativi al completamento della fase di raccolta dati per un aeroporto.

- **to-notifier**

Topic su cui l'**Alert System** pubblica gli eventi che indicano il superamento (o il mancato rispetto) delle soglie configurate dall'utente.

La separazione dei topic consente di:

- mantenere chiari i confini di responsabilità;
- facilitare l'evoluzione futura del sistema;
- ridurre l'accoppiamento tra i servizi.

5.3 Data Collector come Kafka Producer

Al termine di ogni ciclo di raccolta dei dati, il **Data Collector** assume il ruolo di **Kafka Producer**.

Dopo aver:

1. recuperato i dati da OpenSky Network;
2. aggiornato il database;
3. calcolato il numero di voli in arrivo e in partenza;

il Data Collector invia un messaggio sul topic to-alert-system contenente le seguenti informazioni principali:

- e-mail dell'utente;
- codice dell'aeroporto;
- timestamp della raccolta;
- numero di voli in arrivo;
- numero di voli in partenza.

Questo evento rappresenta una **notifica di completamento della fase di aggiornamento** e non contiene alcuna logica di valutazione delle soglie.

5.4 Alert System come Consumer e Producer

L'**Alert System** è un microservizio indipendente che svolge una doppia funzione:

- **Kafka Consumer** del topic to-alert-system;
- **Kafka Producer** del topic to-notifier.

Alla ricezione di un evento, l'Alert System:

1. estrae i dati relativi all'utente e all'aeroporto;
2. recupera dal database i valori di high-value e low-value;
3. confronta i valori di traffico con le soglie configurate;
4. genera uno o più eventi di alert in caso di superamento delle soglie.

Ogni alert prodotto contiene:

- l'e-mail dell'utente;
- l'aeroporto interessato;
- la condizione di superamento (soglia alta o soglia bassa).

Questo servizio incapsula interamente la **logica di business legata alle soglie**, mantenendo il Data Collector privo di responsabilità applicative.

5.5 Alert Notifier System come Kafka Consumer

L'**Alert Notifier System** è l'ultimo elemento della catena asincrona ed è responsabile esclusivamente dell'invio delle notifiche all'utente.

Questo microservizio:

- consuma i messaggi dal topic to-notifier;
- interpreta il contenuto dell'alert;
- invia una notifica tramite protocollo SMTP.

L'uso di Kafka permette all'Alert Notifier di:

- gestire i messaggi in modo sequenziale;
- evitare la perdita di notifiche;
- non dipendere direttamente dal Data Collector o dall'Alert System.

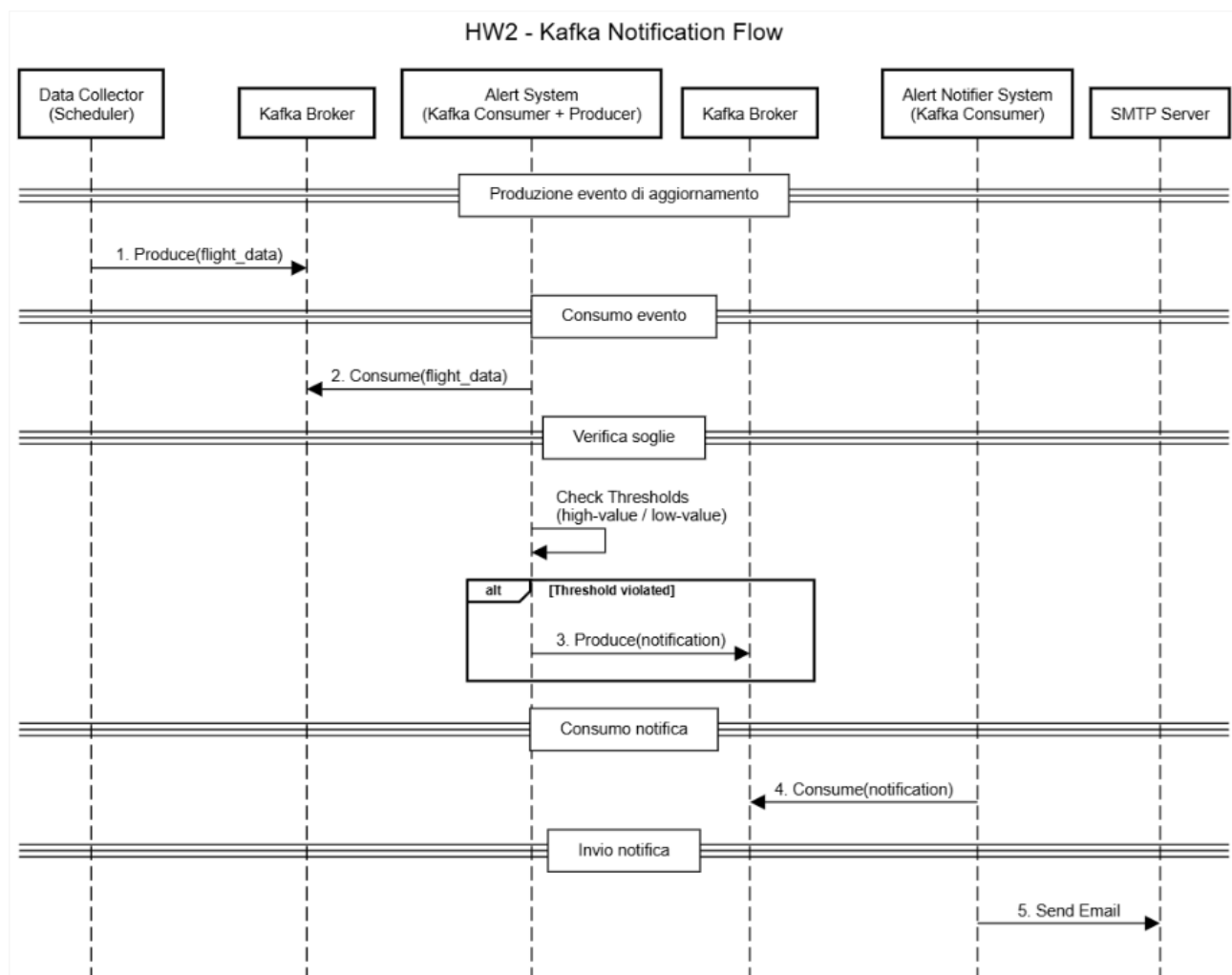
5.6 Diagramma di sequenza del sistema Kafka

Il diagramma mostra il flusso asincrono introdotto in Homework 2 per la gestione delle notifiche.

Al termine della raccolta dei dati, il Data Collector produce un evento sul topic to-alert-system.

L'Alert System consuma l'evento, verifica le soglie configurate dall'utente e, in caso di violazione, pubblica una notifica sul topic to-notifier.

L'Alert Notifier consuma infine l'evento di notifica ed invia una e-mail all'utente tramite SMTP.



5.7 Configurazione del servizio di notifica tramite SMTP

Il servizio Alert Notifier System rappresenta l'ultimo stadio del flusso di notifica asincrona introdotto in Homework 2.

Esso è responsabile della consegna finale delle notifiche agli utenti, tramite l'invio di email utilizzando il protocollo SMTP.

Il servizio agisce come consumer Kafka del topic `to-notifier` e, per ogni evento ricevuto, costruisce ed invia un messaggio di posta elettronica contenente le informazioni relative alla violazione delle soglie configurate dall'utente.

Per garantire flessibilità e separazione tra configurazione e codice applicativo, tutti i parametri SMTP sono forniti al servizio tramite **variabili d'ambiente**, secondo le best practice dei sistemi containerizzati.

Le principali variabili utilizzate sono:

- `SMTP_HOST`: hostname del server SMTP (es. `smtp.gmail.com`)
- `SMTP_PORT`: porta del server SMTP (es. `587`)
- `SMTP_USERNAME`: indirizzo email del mittente
- `SMTP_PASSWORD`: password o *application password* associata all'account SMTP
- `SMTP_USE_TLS`: flag per abilitare la comunicazione cifrata TLS

Questa configurazione consente di:

- cambiare provider email senza modifiche al codice
- evitare l'hardcoding di credenziali sensibili
- semplificare il deployment in ambienti diversi (sviluppo, test, produzione)

Il servizio implementa una gestione esplicita delle principali condizioni di errore legate all'invio delle e-mail, tra cui:

- credenziali SMTP non valide
- server SMTP non raggiungibile
- indirizzo e-mail del destinatario non valido

- errori di rete temporanei

In caso di errore durante l'invio:

- l'eccezione viene intercettata e loggata
- l'applicazione non va in crash
- l'evento Kafka viene considerato consumato, evitando reinvii ripetuti

Questa scelta progettuale è coerente con l'obiettivo di **evitare spam di notifiche** e mantenere il sistema resiliente anche in presenza di errori temporanei.

Conclusioni

Il secondo Homework ha permesso di estendere il sistema sviluppato nel primo elaborato introducendo concetti fondamentali dei sistemi distribuiti moderni, quali resilienza, comunicazione asincrona e decoupling tra componenti.

L'architettura risultante non si limita ad aggiungere nuove funzionalità, ma rappresenta un'evoluzione strutturale del progetto iniziale.

L'introduzione del Circuit Breaker nelle interazioni con l'OpenSky Network ha reso il Data Collector più robusto nei confronti di errori, timeout o indisponibilità del servizio esterno, evitando che tali problematiche possano compromettere la stabilità complessiva dell'applicazione. Questo meccanismo consente al sistema di degradare in modo controllato, migliorandone l'affidabilità.

Un elemento centrale dell'estensione architetturale è l'inserimento di Apache Kafka come message broker, utilizzato per implementare un meccanismo di notifica asincrono.

Il Data Collector, una volta completata la fase di aggiornamento dei dati, produce eventi su un topic Kafka, disaccoppiando completamente la raccolta dei dati dalla logica di notifica. Tale approccio permette di separare le responsabilità dei servizi e di evitare dipendenze sincrone non necessarie.

Kafka è stato progettualmente collocato come componente infrastrutturale esterno rispetto ai microservizi applicativi. Questa scelta riflette un'architettura più vicina a scenari reali di produzione, in cui il message broker rappresenta un servizio condiviso e indipendente dal ciclo di vita dei singoli microservizi.

Questa impostazione consente inoltre di preservare gli eventi anche in caso di riavvio dei servizi consumer e rende il sistema naturalmente predisposto a una futura migrazione verso ambienti

orchestrati come Kubernetes, dove Kafka verrebbe tipicamente gestito come servizio dedicato o cluster separato.

Il flusso di notifica è stato ulteriormente suddiviso in due servizi indipendenti: l'Alert System, responsabile della verifica delle soglie configurate dall'utente, e l'Alert Notifier System, incaricato dell'invio delle notifiche. Questa separazione consente di estendere facilmente il sistema introducendo nuovi canali di notifica senza modificare i componenti esistenti. Nel contesto del progetto è stato implementato l'invio di notifiche tramite SMTP, con una gestione esplicita degli errori di invio e dei casi di configurazione non valida.

Nel complesso, il sistema finale presenta una chiara separazione delle responsabilità, un uso consapevole delle comunicazioni sincrone e asincrone e una struttura modulare che facilita la scalabilità e la manutenibilità.

L'architettura sviluppata soddisfa pienamente i requisiti richiesti dall'Homework 2 e costituisce una base solida e coerente per ulteriori estensioni e per un'eventuale evoluzione verso contesti di produzione più complessi.